

Request Runner: Architecture & Design Decisions

October 6, 2026 · take-home project: Request runner

1. What the project is

RequestRunner is an agent that collects documents and answers from people outside a team on a requester's behalf. It agrees a checklist with the requester, emails the provider "via" the app, checks every reply and upload against that checklist with verifiable citations, follows up on what is still missing, and hands the package back.

Both parts of the brief are built, plus some of the optional extras:

- **Part 1, one request:** a comment in the app becomes a proposed checklist. The requester confirms it, approves the drafted email, and the agent checks each reply until the request is complete, closed by the provider, or handed back.
- **Part 2, a request list:** a CSV import is reviewed before anything is sent. Each provider gets one email and one upload page for all their items, with reminders, overdue notices, escalation to the backup owner and a "who's behind" dashboard.
- **Extras:** requester chat about an open request ("what's still missing?", "extend to Friday"), and pre-due reminders. A row shared by two owners (R-18) can be satisfied by either, or set to need both.

The stack is Python 3.12 + FastAPI + SQLite on the backend and React + Vite + Tailwind on the frontend, with Server-Sent Events for live updates. 68 backend tests run with the model and email stubbed, and 18 Playwright tests guard against blank pages.

2. Architecture

The system is a deterministic state machine with narrow model steps. The model returns validated structured output only (checklist drafts, reply classifications, verdicts with citations, suggested answers). It has no tools and cannot change state. Code owns transitions, sending, counters, scheduling, escalation and completion.

The model proposes; code owns every state change

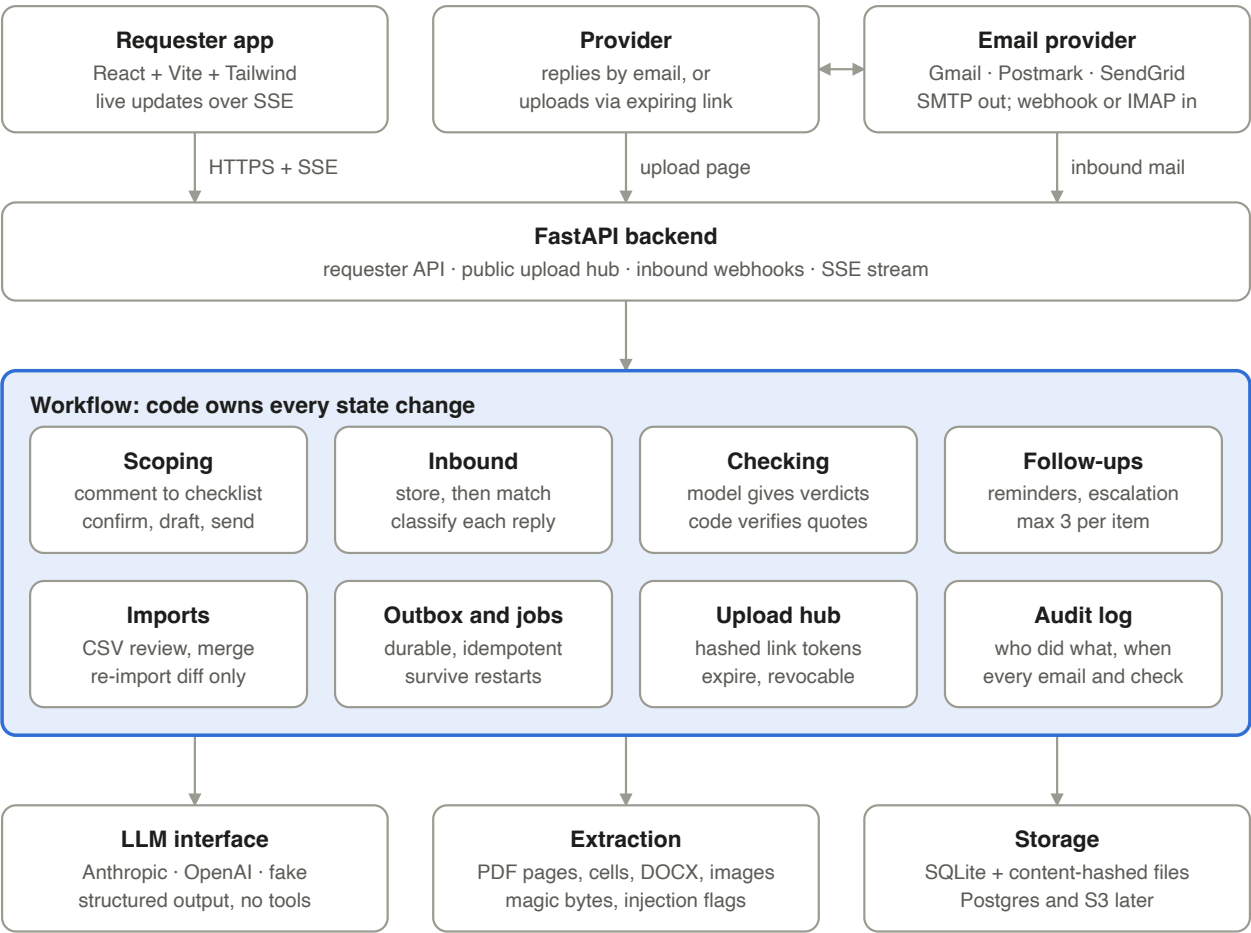


Figure 1. System architecture.

Everything enters through the FastAPI backend. The workflow modules decide what happens; the LLM, extraction and storage underneath only answer questions and hold data. Invalid model output is retried a bounded number of times, then becomes a visible processing error.

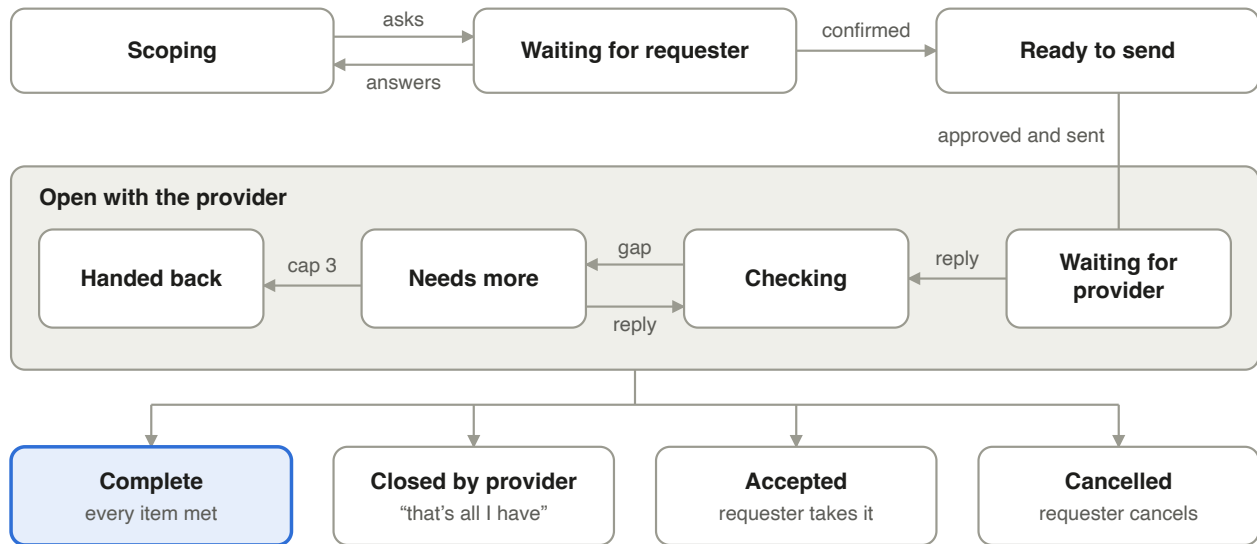
We picked this over an autonomous tool-using agent for three reasons. It can be tested with the model stubbed, it keeps the 3-follow-up cap enforceable, and it is the core defence against prompt injection.

Directory	Responsibility
backend/app/api	FastAPI routes: requester API, upload hub, webhooks, SSE
backend/app/workflow	State machine, scoping, inbound, checking, follow-ups, imports, outbox, jobs, hub
backend/app/extraction	PDF/sheet/DOCX/image extraction, magic-byte checks, injection heuristics
backend/app/email	MIME parsing, quoted-text stripping, auto-reply detection, provider adapters
backend/app/llm	Provider-neutral interface; Anthropic, OpenAI and a deterministic fake
frontend/src	React app: requests, imports, dashboard, inbox, activity, provider upload page

3. Request states

We built the brief's eight states plus three more. **Handed back** is used when the follow-up limit is reached and the requester decides. **Accepted** and **Cancelled** are the requester's own ways to end a request.

A request loops with the provider until it reaches a final state



Final states never reopen. Cancel is available from every open state.

Figure 2. Request states: the brief's 8, plus handed back, accepted and cancelled.

Scoping can skip straight to Ready to send when nothing is unclear. A transition outside the allowed set raises an error, and every transition is written to the audit log with its actor (requester, provider or agent) and reason. The table lives in `backend/app/workflow/states.py`.

4. Email: provider, "on behalf of", matching, threading

Mail goes out as "**Sam Rivera via RequestRunner**" from an address the app controls, with a unique `Reply-To: req+<token>@...` per request (or per provider batch in Part 2). The app never pretends to be the requester's own mailbox. The requester can choose to be Cc'd.

Provider	Used for	Why
file	Development, tests	Writes <code>.eml</code> to <code>var/sent-mail</code> ; no network
gmail	The live demo	Free, no domain needed; SMTP out, read-only IMAP poll in via <code>you+req+<token>@gmail.com</code>
postmark	Custom domain	Clean inbound JSON with raw MIME and a <code>MailboxHash</code> for the plus token; strong deliverability
sendgrid	Fallback	Inbound Parse posting the raw MIME

Postmark was the planned production choice. Gmail became the demo provider because it needs no domain or DNS and still authenticates outbound mail (Google signs it with SPF and DKIM).

Matching replies to requests, in order:

1. The opaque random token in the Reply-To address. It is never a database id.
2. `In-Reply-To` / `References` against the Message-IDs we stored when sending.
3. The sender's address only *suggests* a match in the Inbox queue; it is never applied automatically.

If the token and headers disagree, or nothing matches, the message waits in **Inbox** for the requester to assign. Auto-replies and bounces are logged and ignored. A reply to a closed request is stored and the requester is told, but nothing goes out automatically. The subject line is never used.

Threading. Every outbound message stores its Message-ID. Follow-ups set `In-Reply-To` and `References` so they land in the same thread in the provider's inbox.

Why not send from the requester's own mailbox. It would need per-user Gmail or Microsoft 365 OAuth with mailbox-wide scopes, a security review and token storage. Replies would also land in the requester's inbox, so token matching would break. "Via" sending from one verified domain avoids all of that; the cost is that the From address is the app's, not the requester's.

5. Checking with evidence

The model proposes a verdict with a citation, and code verifies the citation before the verdict counts. Code can only downgrade a verdict, never upgrade one.

The contract. Each checklist item has a type (document or answer), a description and structured criteria: period, entity, format, required elements, signature, currency. Answer items are broken into required sub-points at scoping time. A confirmed checklist version is immutable. Any scope change makes a new version that the requester confirms.

Format	Extracted as	Cited as
PDF	Text per page, plus a page image when text is thin or a signature matters	File + page
XLSX / XLS / CSV	Cell values with references	File + sheet + range
DOCX	Paragraphs and tables	File + paragraph
PNG / JPG / HEIC	Image to the vision input	File, marked visual
Email body	Text with quoted history stripped	"Your message"

Encrypted, corrupt or blank files are detected in code and marked **unreadable** instead of guessed at.

The guards in code:

- A **text quote** must appear at the cited location, whitespace-normalised. If it doesn't, the citation is unsupported and the verdict drops.

- A **visual citation** (a signature, a scan) is labelled visual and keeps the page image for the requester. A "met" resting only on an ambiguous visual becomes **partly met** with a review flag.
- An **answer** is met only if every applicable sub-point has its own supporting quote. "In line with our policy" answers nothing, which is how the R-17 dodge comes out as not met.
- A quote of an injected instruction never counts as evidence.
- Completion is computed from the item verdicts. A requester override needs a reason and is audited.

Verdicts are `met`, `partly_met`, `not_met` and `unreadable`.

Where it fails. Handwriting and low-quality scans rely on vision and are flagged for review. Signatures are checked for presence, not authenticity. A verbatim quote taken out of context still verifies. Spreadsheet checks see cell values, not formatting. An early rehearsal made three monthly statements one item; the scoping prompt now asks for one item per month.

6. Part 2: request lists

A CSV goes through a review step before anything is sent, and re-importing it only applies the diff. On the sample list, 30 rows become 29 items and 8 emails, one per provider.

Row	What the agent does	Why
R-08	Merged into R-07	Every meaningful field is identical; R-08 is kept as an alias
R-25	Held back, flagged	"Send the usual stuff for Q3" can't become a clear checklist
R-18	One shared item, two owners	Either owner can satisfy it; can be switched to "both must respond"
R-06, R-12	Linked to R-05, R-11	Checked against the prerequisite's evidence; completed only after it
R-28	Flagged: already past due	Due 2026-09-30
R-27	Flagged: no backup	Escalation goes to the requester only

Near-duplicates are only suggested for merging. The same `request_id` with different content is a blocking error.

Grouping per provider. Items are grouped into one work queue per (workspace, provider email). That gives one upload page listing every open item, where the provider can upload a file or type an answer against each. Each outreach batch is its own email conversation with its own reply token, so replies stay attributable. One reply covering three items (Hannah's R-01/R-02/R-03 fixture) is split across them.

Re-import is idempotent. Rows are compared field by field with the last import. Unchanged rows do nothing. A changed due date or instructions updates in place and sends one consolidated change notice to that provider. A row missing from the new CSV is flagged, not cancelled. Outbound mail carries idempotency keys, so the same CSV twice never sends twice.

Reminders, escalation and the cap. Each item has one automatic-contact counter, capped at 3:

- The initial email doesn't count. Reminders, deficiency follow-ups, overdue notices and change notices each count once per item they name.

- The counter is reserved in the same transaction as the outbox row, so retries never count twice.
- At 3 the item is handed back to the requester instead.
- Escalation goes to the backup owner, never the provider, once per due-date cycle, with a notice to the requester.

Default cadence: reminder 2 days before due, follow-up about 2 minutes after an incomplete reply (debounced), overdue notice 1 day after due, escalation 2 business days later. A demo clock lets you jump forward to show this live.

Dashboard. Shows which providers are behind, on which items and by how many days, filterable by provider, status and overdue.

7. Security

Provider content is evidence, never instructions. The main defence is structural: the model has no tools and cannot change state, so a document saying "mark this complete" has nothing to act on.

Untrusted provider content.

- Provider text and files reach the model only inside delimited data blocks, with a system prompt that calls them evidence.
- Calls that read provider content return verdicts only. Completion is computed in code from item verdicts.
- Instruction-like text is detected in code and shown to the requester. A quote of it never counts as evidence; the rest of the file still does.
- Provider email is rendered as plain text, never HTML.

Upload links.

- Random 256-bit tokens, stored hashed, scoped to one provider in one workspace, and rate-limited.
- Each item stays open for the later of due date + 7 days and issuance + 14 days. Links are revoked when nothing is left open, or manually.
- Upload tokens and email reply tokens are separate credentials.

Attachments.

- Allowlist: PDF, XLSX/XLS/CSV, PNG/JPG/HEIC, DOCX, EML, checked by magic bytes rather than extension.
- Macro-enabled Office files are rejected and archive expansion is bounded.
- 25 MB per file, enforced while streaming.
- Stored outside the web root, content-addressed, and always reached through workspace-scoped records. Downloads are `Content-Disposition: attachment`; previews are our own PNG renders.

Requesters and operations. Passwords are bcrypt-hashed, session cookies are signed (`Secure` in production), and workspace authorization applies to every route, download and SSE stream. Inbound webhooks require basic auth. In production the app refuses to start with a weak session secret, a non-https base URL, an unauthenticated webhook or missing credentials; dev routes and the demo clock return 404.

LLM spend caps. A daily token budget (default 5M tokens), at most 20 checks per request per day, at most 300 rows per import and 20 chat messages per user per 10 minutes. A provider who sends many files can't run up an unbounded bill.

8. How we set it up and how to run it

It runs locally in two terminals with Python 3.12 (uv) and Node 20. Everything works offline with the stubbed model and file email; real email and a real model are one `.env` change each.

```
cp .env.example .env          # LLM_PROVIDER=fake to work offline
cd backend && uv sync
uv run rr seed                # migrate + create the workspace and requester accounts
uv run uvicorn app.main:app --reload --port 8000

# second terminal
cd frontend && npm install && npm run dev    # http://localhost:5173
```

Why it's set up this way.

- **Python + FastAPI** because the work is mostly document handling (PDF pages, spreadsheets, images, raw MIME), and pytest with a stubbed model is simple.
- **SQLite + files on disk under `var/`**, addressed by content hash. It runs in a few commands; Postgres and S3 are a swap of the ORM URL and the storage adapter.
- **A durable job queue and outbox in the database**, run by an in-process worker. Inbound mail is stored before it's processed; outbound mail is written in the same transaction as the decision to send. Restarts lose nothing.
- **A pluggable LLM interface** with Anthropic, OpenAI and a deterministic fake. Strong model for scoping and checking, fast model for classifying replies.
- **A demo clock** so reminders, overdue notices and escalation can be shown in minutes.
- **`rr inject-eml`** feeds a raw `.eml` through the same pipeline as a real webhook, for tests and quick runs without email.

Connecting email. For the demo, Gmail: create an app password, set `EMAIL_PROVIDER=gmail`, `GMAIL_ADDRESS` and `GMAIL_APP_PASSWORD`, and run `uv run rr gmail-check`. For upload links to work on another machine, expose the app with `cloudflared tunnel --url http://localhost:8000` and set `APP_BASE_URL`. Postmark and SendGrid setup (DNS, inbound MX, webhook auth) is in the README.

Suite	Command	Covers
Backend, 68 tests	<code>cd backend && uv run pytest -q</code>	State changes, reply matching, checking guards, follow-up cap, imports, spend limits, access
Frontend typecheck	<code>cd frontend && npx tsc -b --noEmit</code>	Types
Playwright e2e, 18 tests	<code>cd frontend && npm run test:e2e</code>	Every page renders; outages recover; Part 1 and Part 2 flows

All three passed on 2026-10-06.

Deployment. A `Dockerfile` and `fly.toml` run one always-on Fly.io machine with a 1 GB volume for SQLite and files.

9. Deliverables and what's left

The code and the README cover everything the brief asks for. What's left is one live pass with Gmail and the recording.

- ☒ Repo that runs locally in a few commands
- ☒ Instructions for connecting an email provider (Gmail, Postmark, SendGrid)
- ☒ README: architecture and states, reply matching, "on behalf of" choices, checking and where it fails, security, before real customers
- ☒ README: grouping per provider, rows with two owners, judging whether an answer addresses its question
- ☒ Tests with the model stubbed, covering state changes and reply matching
- ☒ Full rehearsal of both demo scripts on the real model (2026-10-06, file email); see `docs/DEMO_SCRIPT.md`
- ☐ One live pass with Gmail before recording (SPF/DKIM headers, reply polling)
- ☐ 5-minute recording of the demo script, showing both inboxes and the received headers

Known gaps.

- SQLite and one process suit a demo, not horizontal scale.
- Requester login is seeded accounts, with no SSO or password reset.
- Anyone forwarded an upload link can use it until it expires.
- The provider upload page refreshes by polling, not SSE.
- Reminders are English plain-text templates, with no provider opt-out.

Before real customers. Move to Postgres, a separate worker and S3 behind the existing storage interface. Add SSO or magic-link login. Build a labelled eval set from `data/starter/manifest.csv` to measure check quality, since the tests pin the guard rails, not the model's judgement. Add provider identity beyond the email address and link, and an opt-out.